

Instagram Fake Account Detection Using Machine Learning and Deep Learning

Submitted by:
Deepika Verma

1. Introduction

Social media platforms like Instagram have millions of active users worldwide. However, the presence of fake and spam accounts has become a significant issue. These fake accounts can spread misinformation, perform scams, manipulate engagement metrics, and negatively impact user experience.

Detecting fake accounts manually is inefficient due to the large volume of users. Therefore, Machine Learning and Deep Learning techniques can be used to automatically classify accounts as fake or genuine based on user profile features.

The objective of this project is to build predictive models that can accurately classify Instagram accounts using structured profile data.

2. Problem Statement

The main objective of this project is:

To develop a classification model that can accurately predict whether an Instagram account is fake or genuine based on profile features such as username characteristics, follower count, post count, profile picture presence, and other metadata.

3. Dataset Description

The dataset contains profile-level attributes of Instagram accounts collected in 2019.

Dataset Size:

- Training Data: 576 records
- Test Data: 120 records
- Total Features: 12
- Target Variable: `fake`

Feature Description:

Feature	Description
profile pic	1 if profile picture exists, 0 otherwise
nums/length username	Ratio of numbers in username
fullname words	Number of words in full name
nums/length fullname	Ratio of numbers in full name
name==username	1 if username and name are same
description length	Length of bio
external URL	1 if external link exists
private	1 if account is private
#posts	Number of posts
#followers	Number of followers
#follows	Number of accounts followed
fake	Target variable (1 = Fake, 0 = Genuine)

4. Exploratory Data Analysis (EDA)

EDA was performed to understand the distribution and relationships between features.

Key Observations:

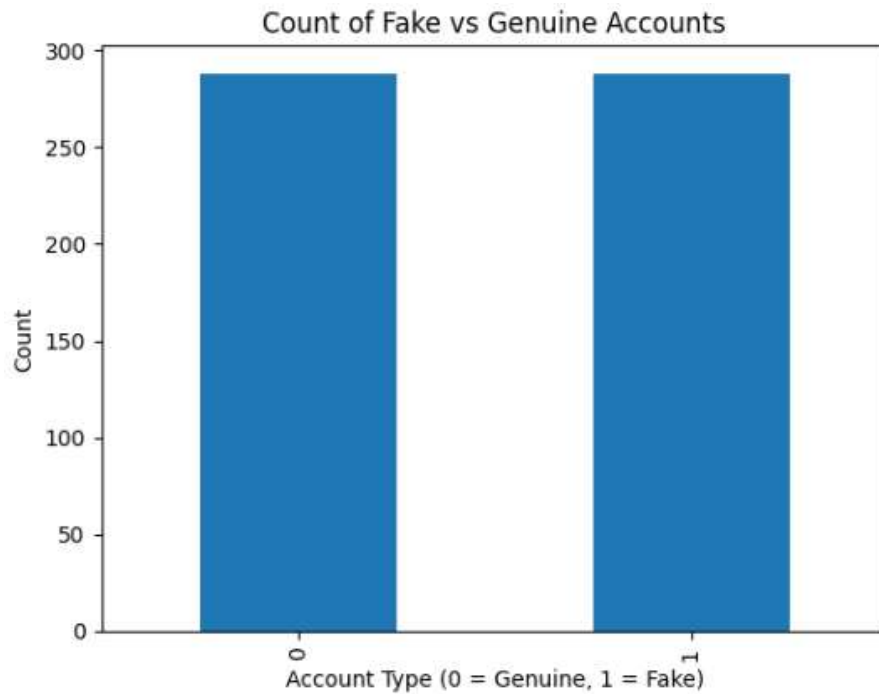
- Fake accounts often lack profile pictures.
- Fake accounts tend to have higher numeric content in usernames.
- Followers and following counts are highly skewed.
- The dataset is balanced:
 - 288 fake
 - 288 genuine

Visualizations Used:

- Count plots (Fake vs Genuine)

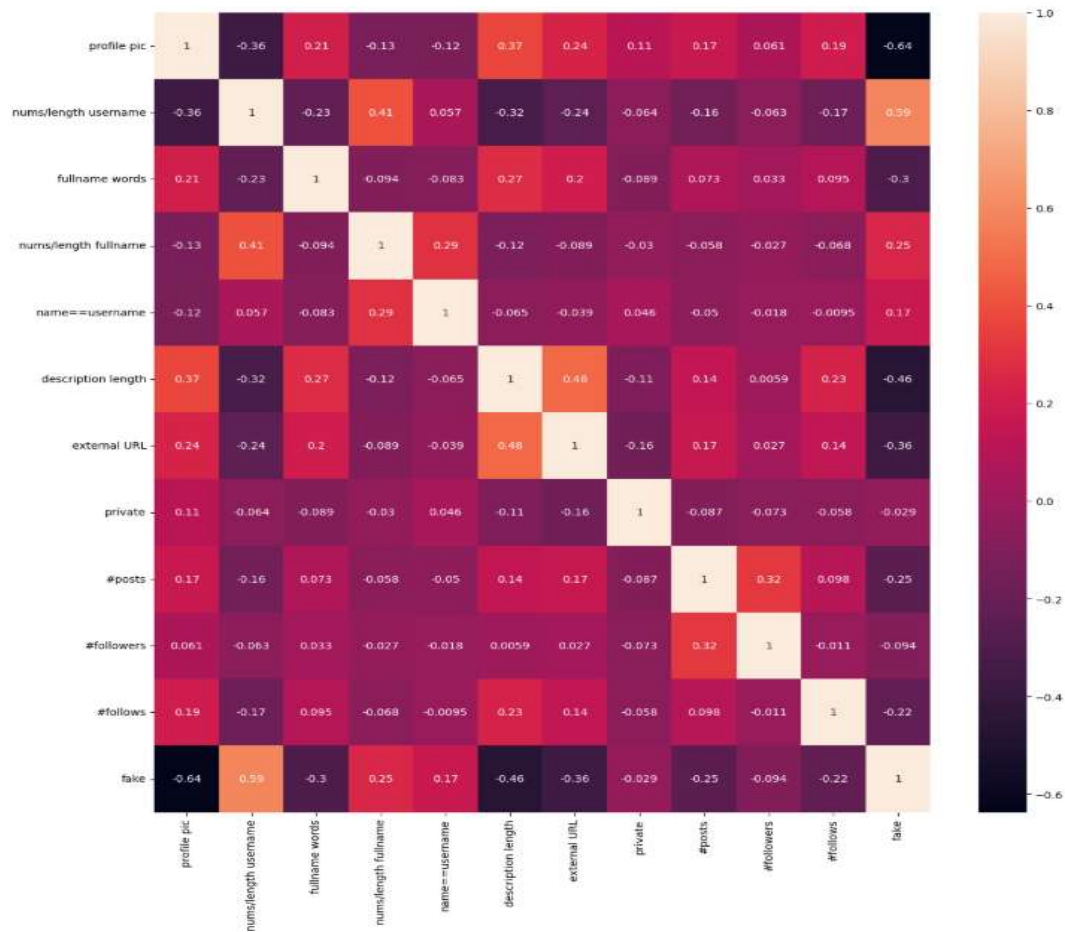
Observation:

- 288 Genuine accounts
- 288 Fake accounts
- Dataset is perfectly balanced (50%-50%)

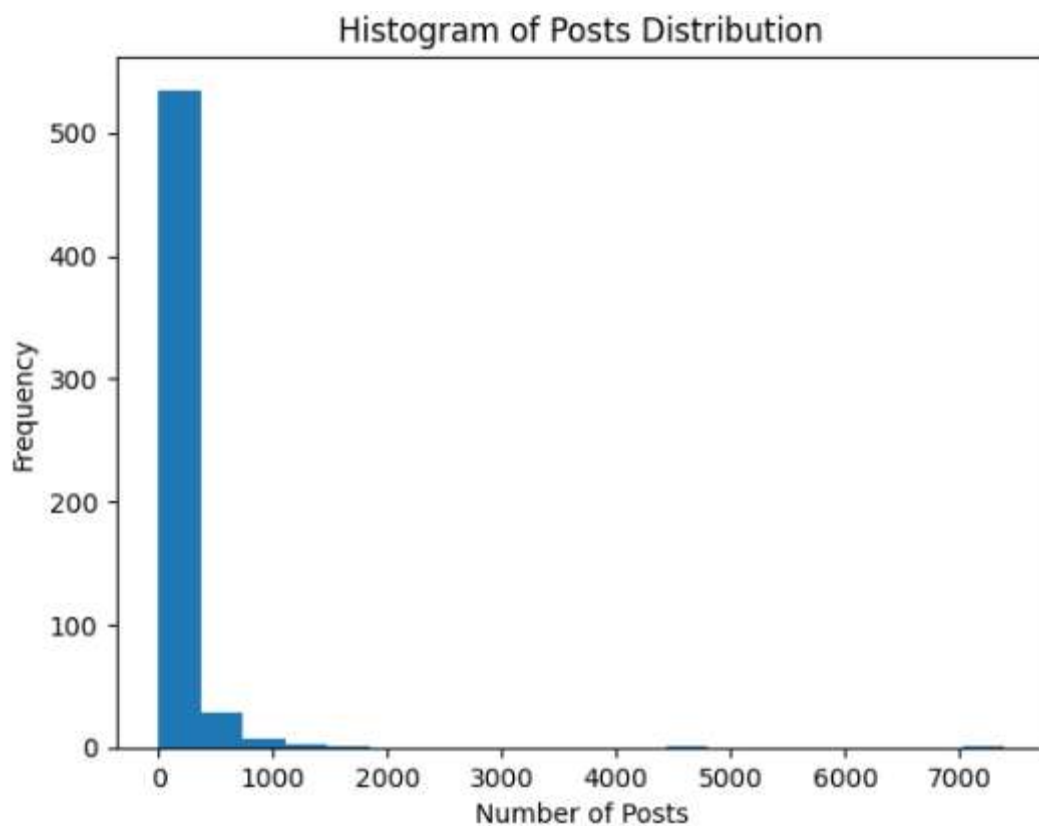


- Correlation heatmap

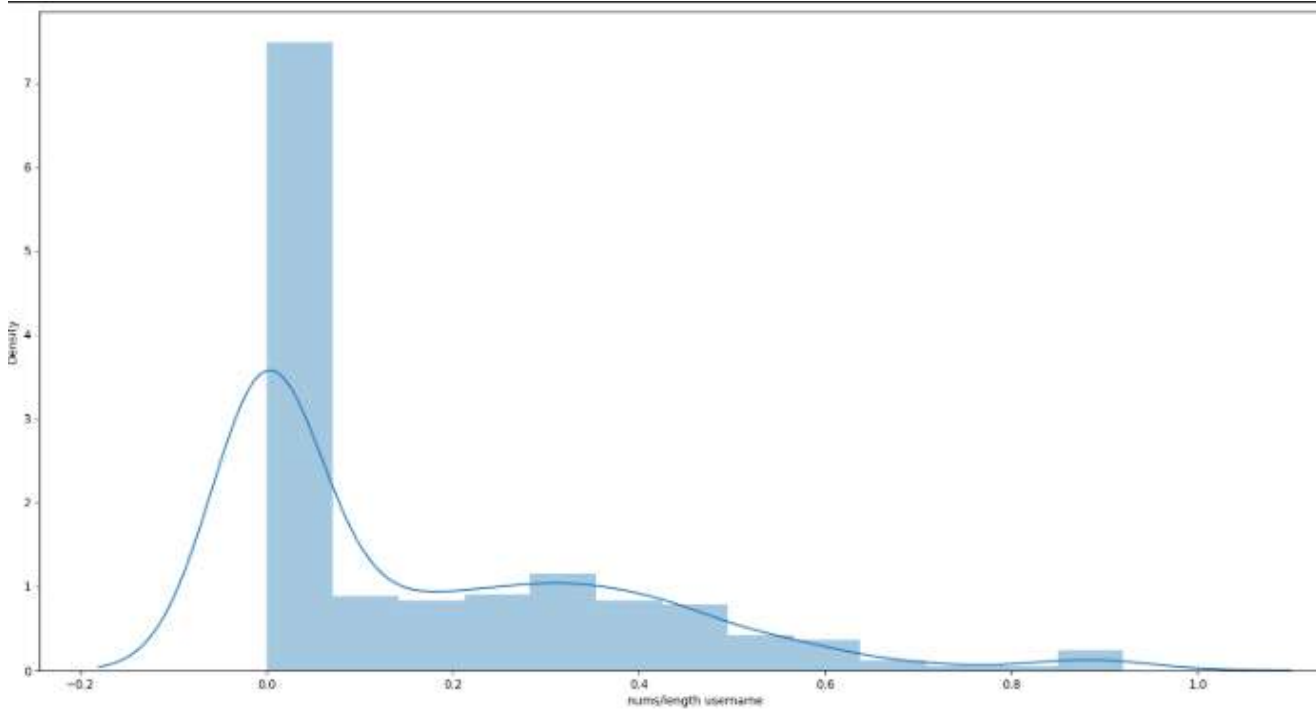
Feature	Correlation	Interpretation
profile pic	-0.63	Strong negative correlation
nums/length username	+0.58	Strong positive correlation
description length	-0.46	Moderate negative
external URL	-0.36	Moderate negative
#posts	-0.24	Weak negative



- Histogram distribution



- Visualizing the length of the user



Observation:

- Genuine accounts tend to have longer descriptions.
- Fake accounts often have empty or very short bios.

EDA helped identify influential features for classification.

5. Data Preprocessing

The following preprocessing steps were performed:

- Checked for missing values (none found)
- Verified data types
- Feature scaling (where necessary)
- One-hot encoding for ANN target
- Train-test split

No major cleaning was required as dataset was well structured.

6. Model Building

Two types of models were implemented:

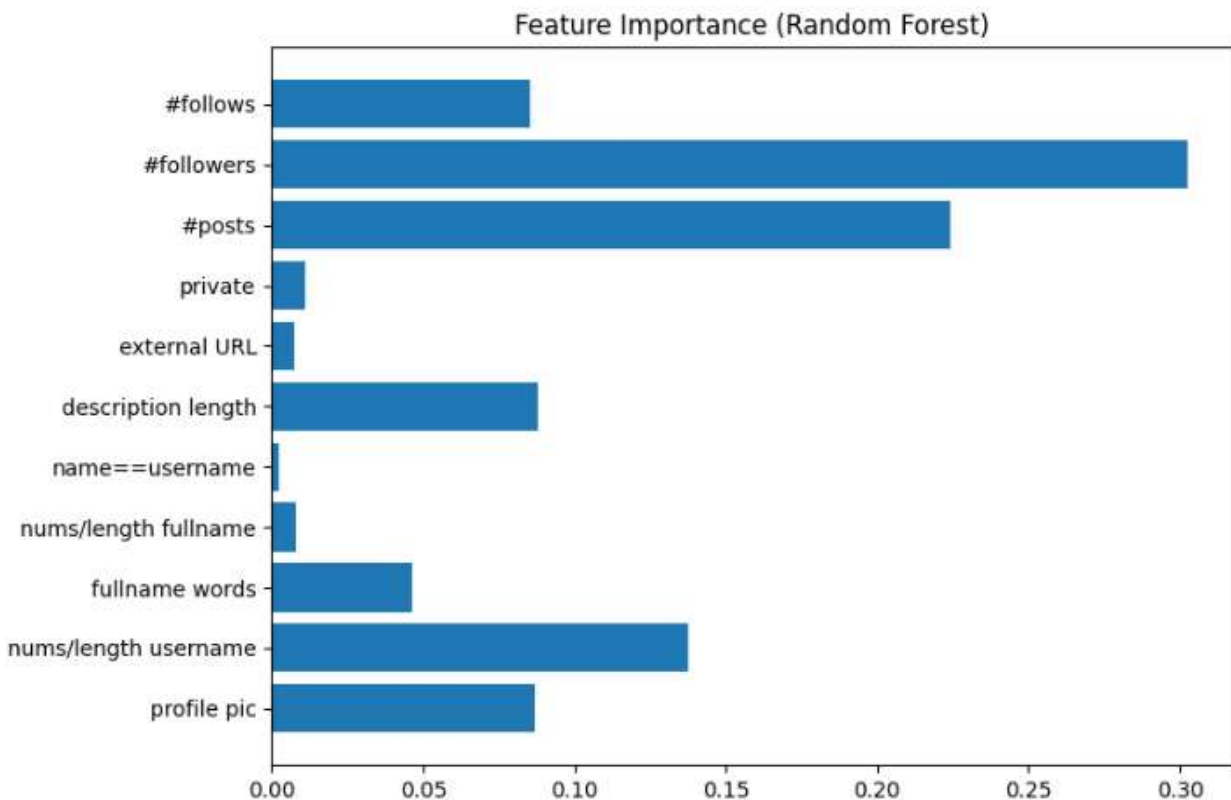
A. Machine Learning Models

1. Decision Tree Classifier

- Simple tree-based classifier
- Accuracy: ~87%

2. Random Forest Classifier

- Ensemble method
- Reduced overfitting
- Improved generalization



The feature importance plot shows which variables contribute most to model decisions.

☐ Most Important Features

1. #followers (~0.30 importance)

- Most influential feature.
- Suggests strong relationship between follower count and target class.
- Likely a key behavioral indicator.

2. #posts (~0.23)

- Second most important predictor.
- Posting activity significantly affects classification.

3. nums/length username (~0.14)

- Usernames with high numeric ratios are strong indicators.
- May signal automated or suspicious accounts.

Random Forest provided better stability compared to Decision Tree.

B. Deep Learning Model (ANN)

A Multi-Layer Perceptron was built using:

- TensorFlow
- Keras

ANN Architecture:

- Input Layer: 11 neurons
- Hidden Layer 1: 50 neurons (ReLU)
- Dropout (0.3)
- Hidden Layer 2: 150 neurons (ReLU)
- Dropout (0.3)
- Hidden Layer 3: 25 neurons (ReLU)
- Dropout (0.3)
- Output Layer: 2 neurons (Softmax)

Loss Function: Categorical Crossentropy

Optimizer: Adam

Epochs: 20

7. Model Evaluation

Evaluation Metrics Used:

- Accuracy
- Precision
- Recall
- F1-Score
- Confusion Matrix
- ROC Curve

Precision

Measures how many predicted fake accounts were actually fake.

Recall

Measures how many actual fake accounts were correctly detected.

F1-Score

Harmonic mean of precision and recall.

	precision	recall	f1-score
0	0.86	0.93	0.90
1	0.93	0.85	0.89
accuracy			0.89
macro avg	0.89	0.89	0.89

Confusion Matrix

Showed strong classification ability with low false positives and false negatives.

ROC Curve Analysis

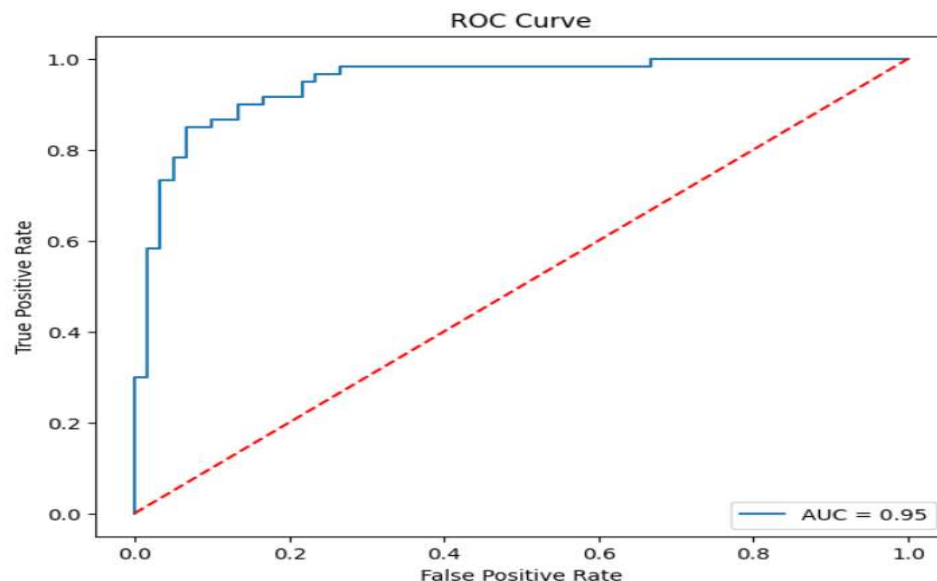
□ Key Observations:

- **AUC = 0.95**
 - This indicates **excellent discriminative performance**.
 - The model correctly distinguishes between the two classes about **95% of the time**.
- The ROC curve is **strongly bowed toward the top-left corner**, meaning:
 - High **True Positive Rate (Sensitivity)**
 - Low **False Positive Rate**
- The curve stays well above the diagonal (random classifier line), showing:
 - The model performs **significantly better than random guessing**.

□ Interpretation:

- The classifier has **very strong predictive power**.
- It achieves high recall without substantially increasing false positives.
- The model is suitable for applications where detecting the positive class is critical (e.g., spam/fake detection).

High AUC value indicated strong class separability.



8. Results

- Decision Tree Accuracy: ~87%
 - Random Forest Accuracy: Higher and more stable
 - ANN Accuracy: High performance with balanced precision and recall
 - Test Dataset Accuracy: ~94%
-

Artificial Neural Network (ANN) Training Results

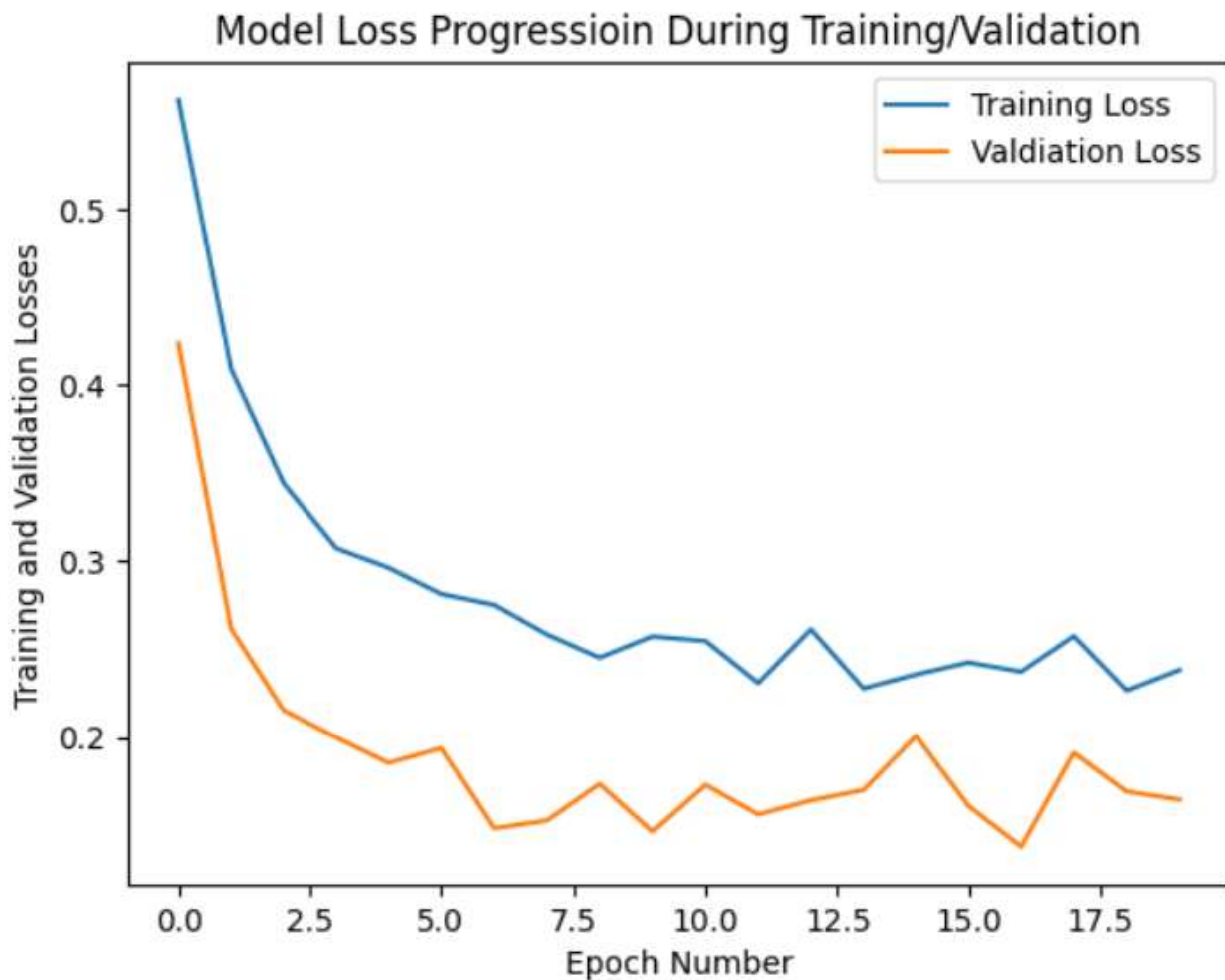
To evaluate the learning performance of the Artificial Neural Network, training and validation loss as well as accuracy were monitored across 20 epochs. The model was implemented using TensorFlow and Keras.

1. Model Loss Progression (Training vs Validation)

□ Graph Description:

The graph shows:

- Blue Line → Training Loss
- Orange Line → Validation Loss
- X-axis → Epoch Number
- Y-axis → Loss Value



Observations:

- Initial training loss started around **0.56**.
- Validation loss started around **0.42**.
- Both losses decreased rapidly within the first 3–5 epochs.
- After approximately epoch 6, the loss values stabilized.
- Final training loss $\approx$ **0.23**
- Final validation loss $\approx$ **0.16–0.17**

Interpretation:

- The rapid drop in early epochs indicates effective learning.
- Validation loss is slightly lower than training loss.
- No major divergence between training and validation curves.
- No significant overfitting observed.

Result:

The ANN model converged successfully and demonstrated stable learning behavior with good generalization capability.

2. Model Accuracy Progression (Training vs Validation)

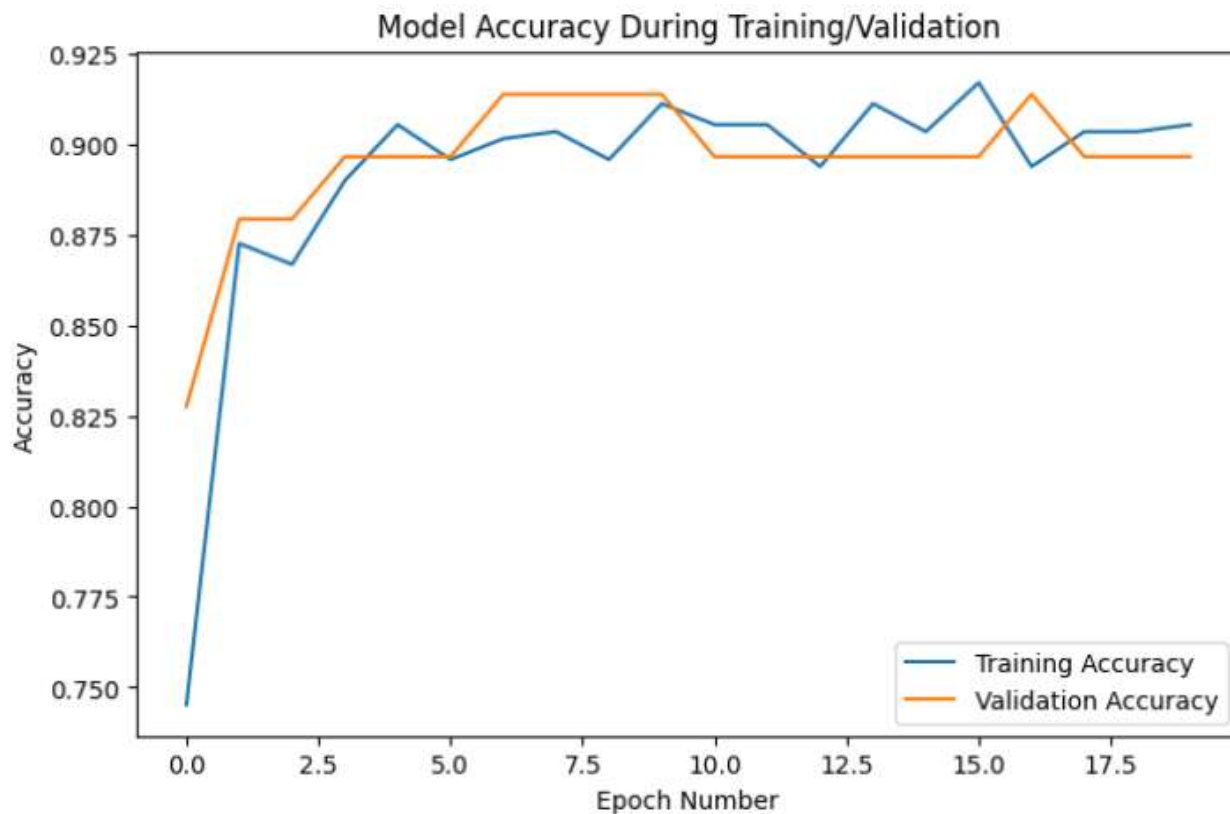
□ Graph Description:

The graph shows:

- Blue Line → Training Accuracy
- Orange Line → Validation Accuracy
- X-axis → Epoch Number
- Y-axis → Accuracy

□ Observations:

- Initial training accuracy started around **74%**.
- Validation accuracy started around **82%**.
- Accuracy improved rapidly during the first few epochs.
- After epoch 5, accuracy stabilized around **90–91%**.
- Final training accuracy $\approx$ **90–91%**
- Final validation accuracy $\approx$ **89–91%**



□ **Interpretation:**

- Model quickly learned underlying patterns.
- Training and validation accuracy curves are very close.
- Minimal gap between training and validation accuracy.
- Indicates good generalization performance.

□ **Result:**

The ANN achieved consistent performance across training and validation sets, confirming that the model does not suffer from overfitting or underfitting.

9. Feature Importance Analysis

Random Forest feature importance analysis revealed:

- profile pic
- nums/length username
- description length
- external URL
- #followers

These features were most influential in detecting fake accounts.

10. Model Saving

Models were saved for deployment:

- Sklearn models saved using Pickle (.pkl)
- ANN model saved using native Keras format (.keras)

This allows reuse without retraining.

11. Conclusion

This project successfully developed machine learning and deep learning models to detect fake Instagram accounts.

The ANN model achieved high accuracy and balanced performance metrics, demonstrating its effectiveness in classification tasks.

Feature analysis revealed that username structure, profile picture presence, and engagement metrics are strong indicators of fake accounts.

The system can be further deployed as an automated fake detection tool.

12. Future Improvements

- Use larger and updated dataset
 - Apply XGBoost for better performance
 - Deploy as web application
 - Real-time Instagram API integration
 - Use NLP on bio descriptions
 - Handle class imbalance for real-world data
-

13. Tools & Technologies Used

- Python
- Pandas

- NumPy
- Seaborn
- Matplotlib
- Scikit-learn
- TensorFlow
- Keras
- Google Colab